

Quest App — Observability Assignment Report

Sarah
Habib University
Enterprise Software Development, Fall 2026
Assignment 1: Observability (Individual)

22 September 2026

Contents

1	Part A: Working Project	3
1.1	API	3
1.2	Trying it out	3
1.3	Limitations (stated honestly)	4
1.4	Environment	4
2	Part B: Metrics (Prometheus and Grafana)	6
2.1	Metrics table	6
2.2	Grafana dashboard	6
2.2.1	p95 and the time window	8
2.2.2	Why the histogram buckets were changed	9
2.2.3	The Summary metric's limitation	9
2.2.4	Self-explored panel: Completion Rate (%)	9
2.2.5	Naming the Node Exporter machine	10
2.2.6	Why the disk panel uses /var/lib	10
2.3	Load testing	10
3	Part C: Logging (Filebeat, Elasticsearch, Kibana)	11
3.1	What is logged	11
3.2	The Filebeat pipeline	11
3.3	Log retention	11
3.4	Restart-survival test	12
3.5	Evidence: one log line, end to end	12
3.6	Kibana queries (saved to logging/kibana-queries.md)	13
4	Part D: Architecture	15
4.1	Architecture diagram	15
4.2	What happens if a component stops	16
4.3	Follow one metric through the whole system: <code>http_requests_total</code>	16
4.4	Follow one log through the whole system: the “quest not found” line	17
4.5	Things not yet fully understood	17

5	Part E: Experiments	18
5.1	E1: Reproduce a Problem (10 marks)	18
5.1.1	Design	18
5.1.2	Prediction (made before running)	18
5.1.3	Stage 1: Baseline (fault off)	18
5.1.4	Stage 2: Fault on	19
5.1.5	Stage 3: Recovery	21
5.1.6	E1 conclusion	21
5.2	E2: Cardinality Explosion (5 marks)	22
5.2.1	Setup	22
5.2.2	Results	22
5.2.3	Cost-at-scale explanation	23
6	Optional Addition: One-Page Web UI	24
7	README, Cleanup, and Submission	25
7.1	Starting and using the project	25
7.2	Safely cleaning up	25
7.3	Submission checklist	25
7.4	AI assistance	25

1 Part A: Working Project

Quest App gives a person a random small “side quest” (for example, “Take a 20-minute walk with no phone”). The person can then mark it *completed*, *skipped*, or *abandoned*.

- **Problem:** people stuck in routine want small, low-effort prompts to break out of it.
- **Users:** anyone who wants a nudge; in this demo, the student and course staff calling the HTTP API directly, or using the small optional web UI described in Section 6.
- **Solution:** a FastAPI service with an in-memory quest pool and quest instances. Every quest lifecycle event and every HTTP request is measured (Prometheus) and logged (structured JSON, shipped to Elasticsearch).

1.1 API

Method & Path	Purpose
GET /quest/new	Get a new random quest
POST /quest/{instance_id}/complete	Mark a quest completed
POST /quest/{instance_id}/skip	Mark a quest skipped
POST /quest/{instance_id}/abandon	Mark a quest abandoned
GET /quest/status	List all quest instances
GET /metrics	Prometheus exposition endpoint
GET /	Health / hello endpoint
GET /ui/	Optional one-page web UI

1.2 Trying it out

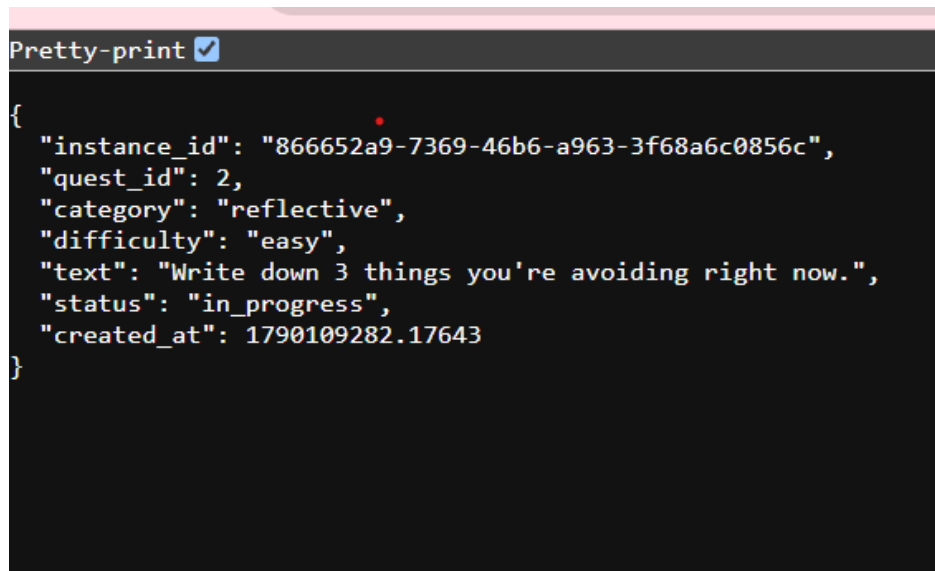
```
docker compose up -d
```

Then open <http://127.0.0.1:8000/quest/new> in a browser, or:

```
curl http://127.0.0.1:8000/quest/new
curl -X POST http://127.0.0.1:8000/quest/{instance_id}/complete
```

A failure example (returns 404):

```
curl -X POST http://127.0.0.1:8000/quest/fake-id/complete
```



```

Pretty-print ☒
{
  "instance_id": "866652a9-7369-46b6-a963-3f68a6c0856c",
  "quest_id": 2,
  "category": "reflective",
  "difficulty": "easy",
  "text": "Write down 3 things you're avoiding right now.",
  "status": "in_progress",
  "created_at": 1790109282.17643
}

```

A successful GET /quest/new response, showing the JSON body (instance_id, category, difficulty, text).

1.3 Limitations (stated honestly)

- Quests live **in memory only** — a restart of the app loses all quest state.
- The core assignment does not require a UI; the API is the primary interface. A small optional UI was added afterwards (Section 6) but is not required for grading.

1.4 Environment

Windows laptop, VS Code with a cmd terminal (not PowerShell), Docker Desktop. The full stack is 7 services defined in one `docker-compose.yml`:

Service	Port	Notes
quest-app	8000	The FastAPI app itself, also serves /ui/
Prometheus	9090	5s scrape interval
Grafana	3000	default login admin/admin
Node Exporter	9100	measures the Docker Desktop WSL2 Linux VM
Elasticsearch	9200	version 8.15.0, named volume esdata
Kibana	5601	version 8.15.0
Filebeat	—	version 8.15.0, reads Docker's json-file logs

```
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	S
elasticsearch	docker.elastic.co/elasticsearch/elasticsearch:8.15.0	"/bin/tini -- /usr/l..."	elasticsearch	25 hours ago	U
p 24 hours	0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp				
filebeat	docker.elastic.co/beats/filebeat:8.15.0	"/usr/bin/tini -- /u..."	filebeat	25 hours ago	U
p 24 hours					
grafana	grafana/grafana:latest	"/run.sh"	grafana	3 days ago	U
p 26 hours	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp				
kibana	docker.elastic.co/kibana/kibana:8.15.0	"/bin/tini -- /usr/l..."	kibana	25 hours ago	U
p 25 hours	0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp				
node-exporter	prom/node-exporter:latest	"/bin/node_exporter ..."	node-exporter	2 days ago	U
p 26 hours	0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp				
prometheus	prom/prometheus:latest	"/bin/prometheus --c..."	prometheus	3 days ago	U
p 15 hours	0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp				
quest-app	quest-app-quest-app	"unicorn app.main:ap..."	quest-app	15 hours ago	U
p 15 hours	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp				

```
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app>
```

Output of `docker compose ps` showing all 7 services `Up`, and confirming Elasticsearch, Kibana, and Filebeat are all version 8.15.0.

Important design fact: Prometheus and Grafana have **no persistent volumes** — their history and dashboard configuration are lost on `docker compose down`. This is exactly why the Grafana dashboard is exported to JSON in `monitoring/dashboards/` as a backup. Elasticsearch's `esdata` volume does persist across `down` (but not `down -v`). Docker's own container log files rotate at $3 \text{ files} \times 10 \text{ MB}$.

2 Part B: Metrics (Prometheus and Grafana)

2.1 Metrics table

All four Prometheus metric types are used. Metrics marked **Business** track quest activity itself (what the app is for); metrics marked **App** track the health and performance of the service delivering that activity.

Metric	Type	Unit	Labels	Recorded where / purpose
quests_generated_total	Counter	quests	category	get_new_quest; Business
quests_completed_total	Counter	quests	category, difficulty	_resolve_quest; Business
quests_skipped_total	Counter	quests	category	_resolve_quest; Business
quests_abandoned_total	Counter	quests	category	_resolve_quest; Business
quests_in_progress	Gauge	quests	category	inc. in get_new_quest, dec. in _resolve_quest; Business
quest_generation_latency_seconds	Histogram	seconds	none	get_new_quest; buckets 10 μ s–1 s; App
quest_time_to_resolve_seconds	Summary	seconds	status	_resolve_quest; how long people take; Business
http_requests_total	Counter	requests	method, endpoint, status	middleware track_requests; App
http_request_duration_seconds	Histogram	seconds	method, endpoint	middleware; buckets 1 ms–2.5 s; App
node_*	various	various	instance, mode, mountpoint, device	Node Exporter, machine metrics; Machine

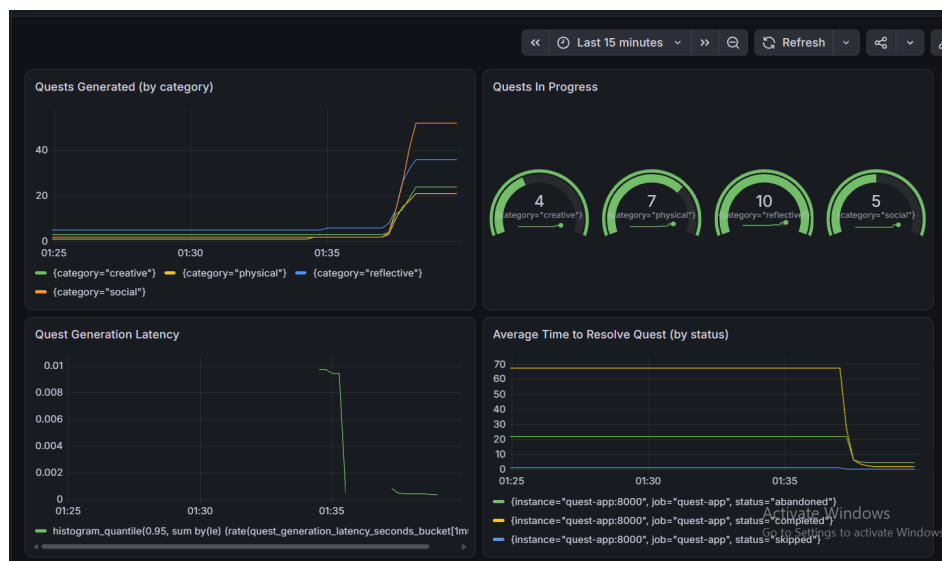
Design choice: the `endpoint` label always uses the *route template* (e.g. `/quest/{instance_id}/complete`), never the real URL. This avoids a cardinality explosion — a principle demonstrated deliberately, in isolation, in Part E2.

2.2 Grafana dashboard

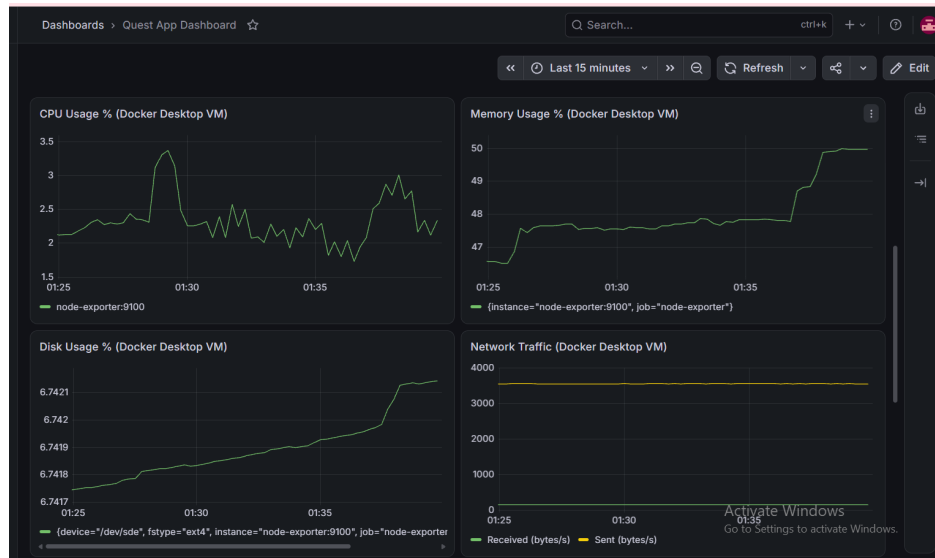
Dashboard name “**Quest App Dashboard**”, uid `ad8j5x6`, built and tested on Grafana 13.2.2. It has 11 panels: 10 built during the core assignment work, plus one self-explored panel (Section 2.2.4). The full panel definitions and queries are exported as JSON in `monitoring/dashboards/`.

Panel	Query
Quests Generated (by category)	<code>quests.generated.total</code>
Quests In Progress	<code>quests.in.progress</code>
Quest Generation Latency (p95)	<code>histogram_quantile(0.95, sumby(1e)(rate(quest_generation_latency_seconds_bucket[\$_rate_interval])))</code>
Avg. Time to Resolve Quest	<code>quest.time.to.resolve.seconds.sum/quest.time.to.resolve.seconds.count</code>
CPU Usage %	<code>100-(avgby(instance)(rate(node_cpu_seconds_total{mode="idle"}[1m]))) * 100</code>
Memory Usage %	<code>100*(1-(node_memory_MemAvailable.bytes/node_memory_MemTotal.bytes))</code>
Disk Usage %	<code>100-(node_filesystem_avail.bytes{mountpoint="/var/lib"}/node_filesystem_size.bytes{mountpoint="/var/lib"}*100)</code>
Network Traffic (Received / Sent)	<code>sumby(instance)(rate(node_network_receive_bytes_total{device!="lo"}[1m])) (Received) and sumby(instance)(rate(node_network_transmit_bytes_total{device!="lo"}[1m])) (Sent) --- one panel, two series</code>
HTTP Requests / s (by status)	<code>sumby(status)(rate(http_requests_total[1m]))</code>
HTTP p95 Response Time	<code>histogram_quantile(0.95, sumby(1e, endpoint)(rate(http_request_duration_seconds_bucket[1m])))</code>

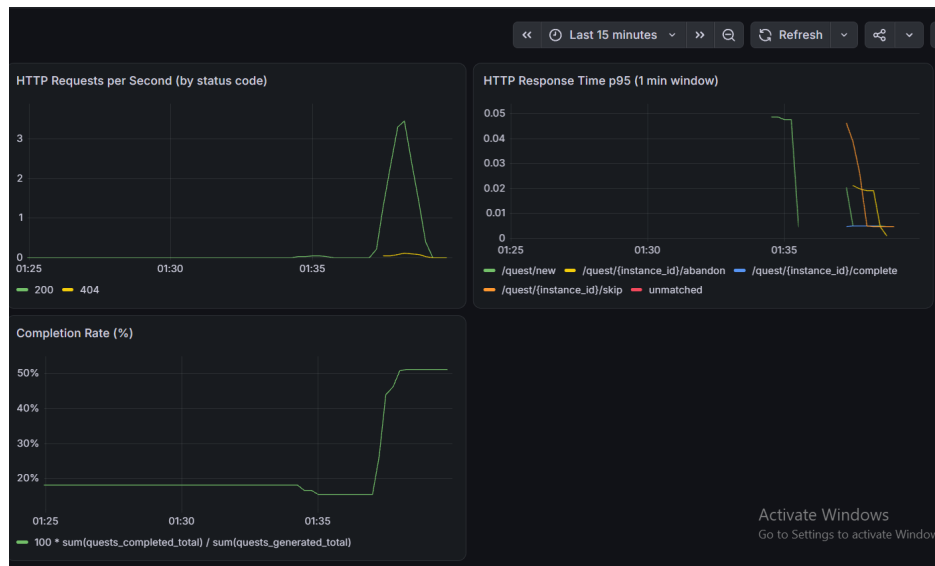
The full dashboard does not fit on one screen at a readable size, so it is captured here in three parts, scrolling top to bottom, all taken during the same live load-test run so the panels show real traffic rather than empty placeholders.



Top of the dashboard: Quests Generated (by category), Quests In Progress, Quest Generation Latency, and Average Time to Resolve Quest (by status), all populated with live data from a running load test.



Middle of the dashboard: the four Node Exporter machine panels — CPU Usage %, Memory Usage %, Disk Usage %, and Network Traffic (Received/Sent as two series on one panel).



Bottom of the dashboard: HTTP Requests per Second (by status code), HTTP Response Time p95, and the self-explored Completion Rate (%) panel.

2.2.1 p95 and the time window

`rate(...[1m])` looks at the last 1 minute of histogram bucket increases (about 12 samples at a 5 s scrape interval); `histogram_quantile(0.95, ...)` then estimates the value that 95% of requests in that window were faster than. The panel shows no data if there was no traffic in the window.

2.2.2 Why the histogram buckets were changed

The Prometheus client’s default buckets start at 5 ms, but quest generation takes microseconds, so every observation landed in the smallest default bucket (`_sum = 0.0`). After switching to custom buckets (10 μ s to 1 s): 13 of 14 requests fell at or below 10 μ s, and Grafana’s p95 read approximately 0.0005–0.0008 s.

2.2.3 The Summary metric’s limitation

Python’s `prometheus_client Summary` only exposes `_count` and `_sum` (so only an average is possible) — percentiles require a Histogram instead. Example observed: one quest resolved as *completed* after about 46 s (`_count = 1`, `_sum  $\approx$  46.3`).

2.2.4 Self-explored panel: Completion Rate (%)

A new panel was added beyond the assignment’s required panels:

```
100 * sum(quests_completed_total) / sum(quests_generated_total)
```

with the unit set to Percent (0–100). It showed approximately 48–56% depending on the traffic window. Grafana displays a warning here (“Selected metric is a counter, consider rate()”); this is a false positive for this specific case — the warning is meant for graphing raw counter growth over time, but here two cumulative counters are being divided into a ratio, which is the correct and intended use.



Close-up (Grafana’s expanded panel view) of the self-explored Completion Rate (%) panel, showing the query and the live chart.

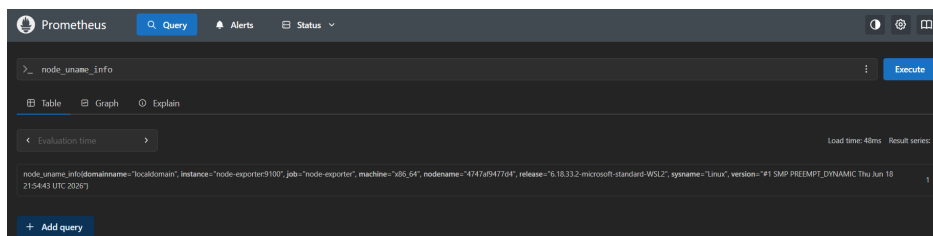
2.2.5 Naming the Node Exporter machine

Prometheus query `node_uname_info` returned:

```
instance="node-exporter:9100", job="node-exporter"
nodename="4747af9477d4"
sysname="Linux"
release="6.18.33.2-microsoft-standard-WSL2"
machine="x86_64"
domainname="localdomain"
version="#1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026"
```

Cross-checked with `docker ps --filter "name=node-exporter"` — the container’s own CONTAINER ID was exactly 4747af9477d4, confirming that `nodename` is simply the Node Exporter container’s own hostname, not a name for the whole virtual machine.

Conclusion: Node Exporter measures the **Docker Desktop WSL2 Linux virtual machine** (kernel 6.18.33.2-microsoft-standard-WSL2, x86_64) running on the student’s Windows laptop — not Windows itself.



Prometheus Table view of the `node_uname_info` query.

2.2.6 Why the disk panel uses `/var/lib`

The Windows C: mount (9p filesystem type, as seen from inside the VM) reports 100% used, which is not meaningful. The VM’s own Docker disk, mounted at `/var/lib` (ext4), reported approximately 6.5% used — the real, usable figure.

2.3 Load testing

`scripts/load_test.py` is a standard-library-only Python load generator with `--duration`, `--rate`, and `--seed` flags (the same seed reproduces the same sequence of outcomes, for repeatable experiments). It sends steady `GET /quest/new` requests, then randomly completes, skips, abandons, leaves open, or hits a deliberately-bad instance ID for each quest returned.

Example run:

```
python scripts\load_test.py --duration 60 --rate 2
```

Result: 229 requests, 120 quests generated (66 completed, 28 skipped, 8 abandoned, 11 left open, 7 bad-id/404s), average 16.4 ms / p95 30.1 ms for `GET /quest/new`.

3 Part C: Logging (Filebeat, Elasticsearch, Kibana)

3.1 What is logged

JsonFormatter in `app/main.py` writes one JSON object per line to stdout. Fixed fields: `time` (UTC, ISO format), `service`, `severity`, `message`, `request_id`. Optional fields (a fixed whitelist): `event`, `method`, `path`, `endpoint`, `status`, `duration_ms`, `instance_id`, `category`, `difficulty`, `quest_status`.

- `request_id` lives in a context variable set by the middleware, so every log line written while handling one request automatically carries that request's ID. The same ID is also returned in the `X-Request-ID` response header.
- Severity: `INFO` for 2xx/3xx, `WARNING` for 4xx, `ERROR` for 5xx.
- Events: `request_completed`, `quest_generated`, `quest_resolved`, `quest_not_found` (WARNING), `quest_already_resolved` (WARNING), `unhandled_exception` (ERROR).
- **No secrets or personal data** are logged — only IDs, categories, difficulty, and timings. The URL-derived `instance_id` is capped at 64 characters and `path` at 100, as a small defensive measure.

3.2 The Filebeat pipeline

1. Reads Docker's per-container log files (`/var/lib/docker/containers/*/*-json.log`) through a read-only bind mount.
2. `add_docker_metadata` adds container name and image.
3. A `drop_event` processor keeps only lines from the `quest-app` container.
4. `decode_json_fields` parses the JSON line into fields under an `app.*` prefix (avoiding clashes with Elasticsearch's standard fields).
5. A second `drop_event` removes non-JSON lines (Uvicorn's own plain-text startup/access lines).
6. A `timestamp` processor sets `@timestamp` from the app's own `app.time` field.
7. Output: Elasticsearch, data stream `quest-app-logs-main`.

Because the app already writes structured JSON, no regex or grok parsing is needed anywhere in this pipeline.

Retention design change: the pipeline originally wrote to a *daily* index pattern (`quest-app-logs-%{+yyyy.MM.d}`) which silently created a brand-new, unmanaged data stream every single day — nothing was ever deleting old logs. This was changed to a **single fixed data stream**, `quest-app-logs-main`, so that one retention setting covers all logs going forward.

3.3 Log retention

Applied via the Elasticsearch API:

```
curl -X PUT "http://localhost:9200/_data_stream/quest-app-logs-*/_lifecycle" \  
  -H "Content-Type: application/json" \  
  -d "{\"data_retention\":\"7d\"}"
```

Response: `{"acknowledged":true}`. Verified with `GET _data_stream/quest-app-logs-*?pretty` — both the old daily stream and the new fixed stream show:

```
"managed_by" : "Data stream lifecycle",
"lifecycle" : { "enabled" : true, "data_retention" : "7d" }
```



```
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> curl "http://localhost:9200/_data_stream/quest-app-logs-*/_lifecycle?pretty"
{
  "data_streams" : [
    {
      "name" : "quest-app-logs-2026.09.21",
      "lifecycle" : {
        "enabled" : true,
        "data_retention" : "7d"
      }
    },
    {
      "name" : "quest-app-logs-main",
      "lifecycle" : {
        "enabled" : true,
        "data_retention" : "7d"
      }
    }
  ]
}
```

The `_data_stream/quest-app-logs-*/_lifecycle?pretty` response, showing both data streams (`quest-app-logs-2026.09.21` and `quest-app-logs-main`) with `"enabled": true` and `"data_retention": "7d"`.

Statement for the report: logs are kept *at least* 7 days, after which Elasticsearch becomes eligible to delete them. Elasticsearch checks lifecycle rules periodically rather than continuously, so actual deletion may lag slightly past the 7-day mark. Docker’s own container log file is separately capped at 3 files × 10 MB.

3.4 Restart-survival test

1. Baseline count: `curl "http://localhost:9200/quest-app-logs-*/_count?pretty" → 20`.
2. `docker compose restart elasticsearch`.
3. Immediately after: `curl: (52) Empty reply from server` — expected, since Elasticsearch takes about a minute to fully boot. This is a good live example of “Elasticsearch is down” for Part D.
4. About a minute later, the same count command returned **20** again.

Conclusion: logs survive an Elasticsearch container restart, because they are stored in the named volume `esdata`, not inside the container itself.

3.5 Evidence: one log line, end to end

Raw line written by the app (a request to complete a nonexistent quest instance):

```
{"time": "2026-09-21T19:58:04.736983+00:00", "service": "quest-app",
"severity": "WARNING", "message": "quest not found",
"request_id": "08d08ced29b7", "event": "quest_not_found",
"instance_id": "fake-id"}
```

As stored in Elasticsearch (trimmed):

```

"_index": ".ds-quest-app-logs-2026.09.21-2026.09.21-000001",
"_source": {
  "@timestamp": "2026-09-21T19:58:04.736Z",
  "app": { "service": "quest-app", "severity": "WARNING",
    "message": "quest not found", "request_id": "08d08ced29b7",
    "event": "quest_not_found", "instance_id": "fake-id",
    "time": "2026-09-21T19:58:04.736983+00:00" },
  "container": { "name": "quest-app" },
  "agent": { "type": "filebeat", "version": "8.15.0" }
}

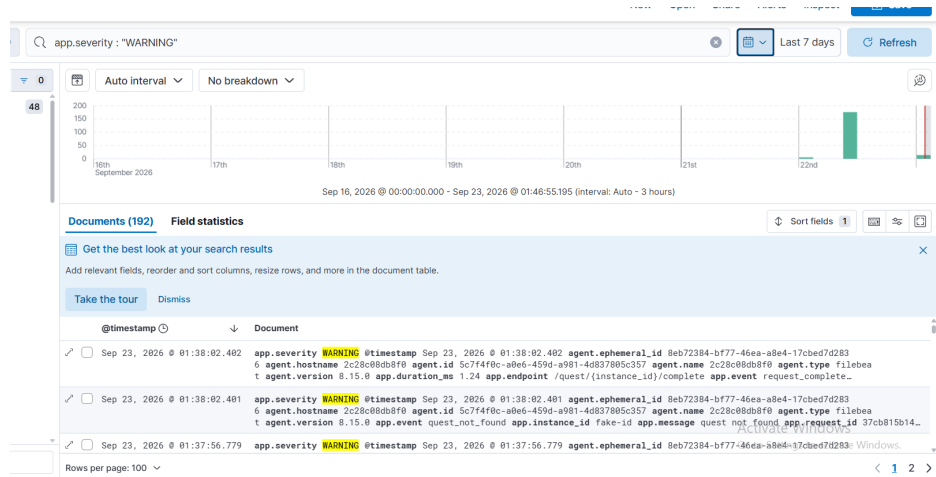
```

Timezone note: Kibana renders in the browser's local time (Pakistan, UTC+5); the raw log and Elasticsearch both use UTC. The same moment appears as 19:58:04+00:00 in the raw log but 00:58:04 in Kibana's Discover view.

3.6 Kibana queries (saved to logging/kibana-queries.md)

Data view: quest-app-logs. Time range used for testing: Last 7 days.

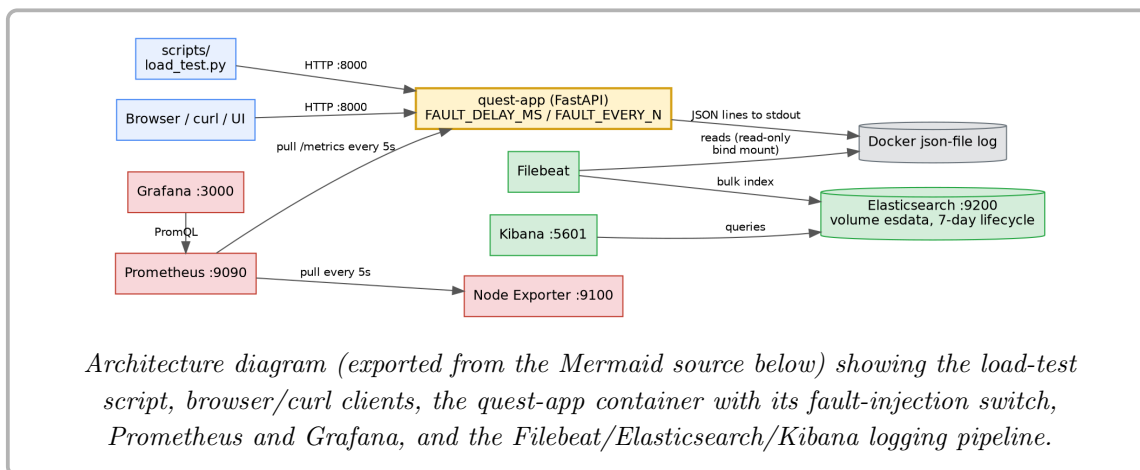
Query	Result	Notes
app.severity : "WARNING"	192 docs	All warning lines accumulated across the whole project's testing to date (load tests, E1, E2); an early spot-check on 21 Sep, before most of this traffic existed, found 2 docs
app.request_id : "08d08ced29b7"	2 docs	Two log lines from one specific failed request
app.event : "quest_generated"	16 docs	Quests handed out
app.status >= 400	1 doc	Only request_completed lines carry status
app.severity : "ERROR"	0 docs	No server errors so far
app.duration_ms >= 400	109 docs	Used in the E1 fault experiment (Part E)



Kibana Discover with `app.severity : "WARNING"` typed in the KQL bar over Last 7 days, showing 192 accumulated documents (a higher count than the original 2-document spot-check, reflecting the traffic generated since by load tests and the E1/E2 experiments).

4 Part D: Architecture

4.1 Architecture diagram



The diagram source (Mermaid), for reference or re-export:

```

flowchart LR
    loadtest["scripts/load_test.py"] -->|"HTTP :8000"| app["quest-app (FastAPI)  
FAULT_DELAY_MS / FAULT_EVERY_N"]
    user["Browser / curl / UI"] -->|"HTTP :8000"| app
    app -->|"JSON lines to stdout"| dlog["Docker json-file log"]
    app -->|"pull /metrics every 5s"| prom["Prometheus :9090"]
    app -->|"pull every 5s"| ne["Node Exporter :9100"]
    graf["Grafana :3000"] -->|"PromQL"| prom
    prom -->|"reads (read-only bind mount)"| fb["Filebeat"]
    fb -->|"bulk index"| es["Elasticsearch :9200  
volume esdata, 7-day lifecycle"]
    kib["Kibana :5601"] -->|"queries"| es
  
```

A standalone `scripts/cardinality_demo.py` on port 9091, scraped by a temporary fourth Prometheus job, was also used for Part E2. It is not part of the app's normal architecture and was removed after the experiment, so it is left out of the main diagram.

Communication pattern: Prometheus *pulls* from the app and Node Exporter; Filebeat *reads* log files and pushes them to Elasticsearch; Grafana and Kibana *query* their respective stores. All 7 core services run in one `docker-compose.yml` and reach each other by service name.

4.2 What happens if a component stops

Component down	Effect
quest-app	Prometheus target flips to <code>up == 0</code> ; gaps appear in all <code>quest_*</code> / <code>http_*</code> metrics; app logs stop entirely; <code>/ui/</code> becomes unreachable.
Prometheus	quest-app itself is completely unaffected (it only exposes <code>/metrics</code> , nothing scrapes it); a gap appears in Prometheus’s own history when it returns; no volume, so any earlier history is lost if the container is removed.
Grafana	Nothing else is affected — Grafana only reads from Prometheus and Elasticsearch. Dashboard configuration is lost if the container is removed (mitigated by the JSON export).
Elasticsearch	Filebeat keeps retrying; logs stay safely in Docker’s own json-file logs until Elasticsearch returns, then get sent — confirmed live (see the restart-survival test, Part C).
Filebeat	App keeps writing logs to Docker as normal; Filebeat’s read-position registry is not on a persistent volume, so if the Filebeat <i>container</i> is recreated (not just restarted) it may re-read from the start and duplicate documents.
Kibana	Data is completely safe (stored in Elasticsearch); only the ability to <i>view</i> logs is lost.
Node Exporter	Only machine-level <code>node_*</code> metrics stop; the app’s own business and HTTP metrics are unaffected.

Live evidence for two of these rows: during the restart-survival test (Part C), Elasticsearch being down for about a minute produced exactly the “Filebeat retries, logs wait” behaviour described above, not just a theoretical claim. During the E1 fault experiment (Part E), the quest-app’s Prometheus target stayed UP the entire time even while 1 in 5 requests was deliberately slowed — a useful contrast against a full outage, where the target would flip to DOWN entirely.

4.3 Follow one metric through the whole system: `http_requests_total`

1. **Code** (`app/main.py`, middleware `track_requests`): after every request (except `/metrics` itself) finishes, `http_requests_total.labels(...).inc()` increments the counter for that method/endpoint/status combination. `endpoint` is the route template, not the raw URL.
2. **Raw exposition** (GET `/metrics`): the running counter appears as a plain-text line, e.g. `http_requests_total{endpoint="/quest/new",method="GET",status="200"} 1027.0`.
3. **Prometheus storage**: scraped every 5s (job `quest-app` in `monitoring/prometheus.yml`), stored as a time series keyed by the full label set.
4. **PromQL query** (used in the “HTTP Requests per Second” panel): `sum by (status)(rate(http_requests_total[1m]))` — the per-second growth rate over the last minute, summed across endpoints, split by status code.
5. **Grafana panel**: renders that query as a line chart. During the E1 load tests this showed a plateau around 5–6 req/s for status 200 while the load script ran, and a thin near-zero line for

404s.

This walkthrough shows the same underlying event taking a different shape at each stage: a Python `.inc()` call → a plain-text exposition line → a stored time series → a rate calculation → a rendered chart.

4.4 Follow one log through the whole system: the “quest not found” line

1. **Code** (`app/main.py`, `_resolve_quest`): calling `POST /quest/{instance_id}/...` with an `instance_id` that does not exist triggers `logger.warning("quest not found", extra={...})`, then a 404 is raised.
2. **JSON written to stdout**: becomes one line, shown in full in Part C, Section 4.
3. **Docker capture**: this stdout line is written into the container’s json-file log at `/var/lib/docker/containers/<id>/<id>-json.log`.
4. **Filebeat**: reads that file, keeps only `quest-app`’s lines, parses the JSON into fields under `app.*`, sets `@timestamp` from `app.time`.
5. **Elasticsearch**: stores the document with the parsed `app.*` fields plus Filebeat’s own metadata (`container.name`, `agent.type`), in the `quest-app-logs-main` data stream.
6. **Kibana search**: `app.request_id : "08d08ced29b7"` in Discover returns 2 documents — this “quest not found” line and the middleware’s paired “request completed” line for the same failed request, joined by a shared `request_id`.

The format changes at every hop: a Python object → JSON text on stdout → a line inside a Docker log file → parsed structured fields in Elasticsearch → a searchable, filterable document in Kibana.

4.5 Things not yet fully understood

Exact timing of log deletion under the 7-day retention policy. I set the retention to 7 days and confirmed Elasticsearch accepted the setting, but I haven’t actually watched a log get deleted — that would mean waiting at least a week and checking back. So there are a few things about the timing I’m honestly not sure about. Is the 7 days counted from when each log was written, or from when its underlying index file was created? How long does it take after the 7 days are up before deletion actually happens — Elasticsearch says it checks periodically rather than constantly, so there’s probably some delay, but I don’t know how much. And since my data hasn’t rolled over into a new index yet (it’s still on the very first one), I’m not sure if deletion can even happen yet, or if it only kicks in once that rollover occurs. If I had more time, I’d test this properly: set retention to something tiny like one minute on a throwaway stream and just watch it happen, instead of guessing based on what the docs say.

5 Part E: Experiments

5.1 E1: Reproduce a Problem (10 marks)

5.1.1 Design

An environment-variable-controlled fault switch was added to `app/main.py`: `FAULT_DELAY_MS` and `FAULT EVERY_N`, both defaulting to "0" (off). Every Nth call to `GET /quest/new` sleeps for the given number of milliseconds before responding. Wired into `docker-compose.yml` via `${FAULT_DELAY_MS:-0}` defaults, so the fault can be toggled with `set` plus `docker compose up -d quest-app` — no rebuild needed.

Design detail: the delay is inserted *before* `start_time` is set for `quest_generation_latency_seconds`, so it inflates the **HTTP-level** metric (`http_request_duration_seconds`, what users and Grafana actually see) but does **not** affect the app’s own internal generation-latency histogram — a deliberate simulation of “something external or slow happened during the request”, not “quest generation itself got slower”.

Fault settings used: `FAULT_DELAY_MS=500`, `FAULT EVERY_N=5` (matching the assignment’s own suggested example).

5.1.2 Prediction (made before running)

`http_request_duration_seconds` has histogram bucket boundaries at 0.5s and 1s. With 1 in 5 requests delayed 500 ms, those requests fall in the (0.5, 1] bucket, and `histogram_quantile(0.95, ...)` interpolates inside it:

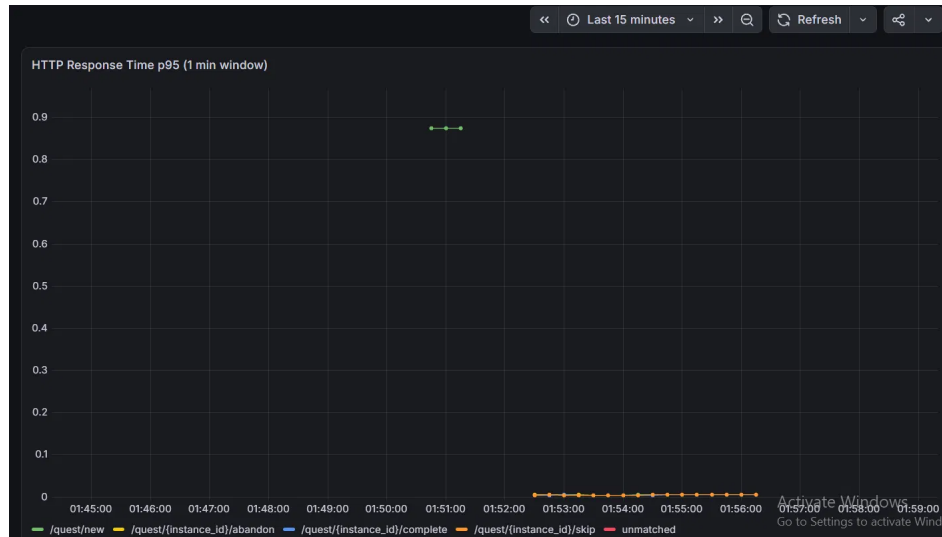
$$0.5 + \left(\frac{0.95 - 0.80}{0.20} \right) \times 0.5 \approx \mathbf{0.875\ s}$$

— not exactly 0.5s. This nuance (percentile estimates depend on bucket boundaries) was stated as the prediction *before* running the experiment, and checked against the actual result below.

5.1.3 Stage 1: Baseline (fault off)

```
python scripts\load_test.py --duration 180 --rate 3 --seed 100
```

- Client-side: 1027 requests, 540 quests generated (290 completed, 120 skipped, 50 abandoned, 53 left open, 27 bad-id). `/quest/new` average **16.8 ms**, p95 **30.0 ms**.
- Grafana server-side p95: approximately **0.004–0.005 s** (4–5 ms) across all endpoints.



Grafana's HTTP p95 latency panel during the baseline stage (flat, near zero).

5.1.4 Stage 2: Fault on

Manual verification before the full test — five sequential `curl` timing checks:

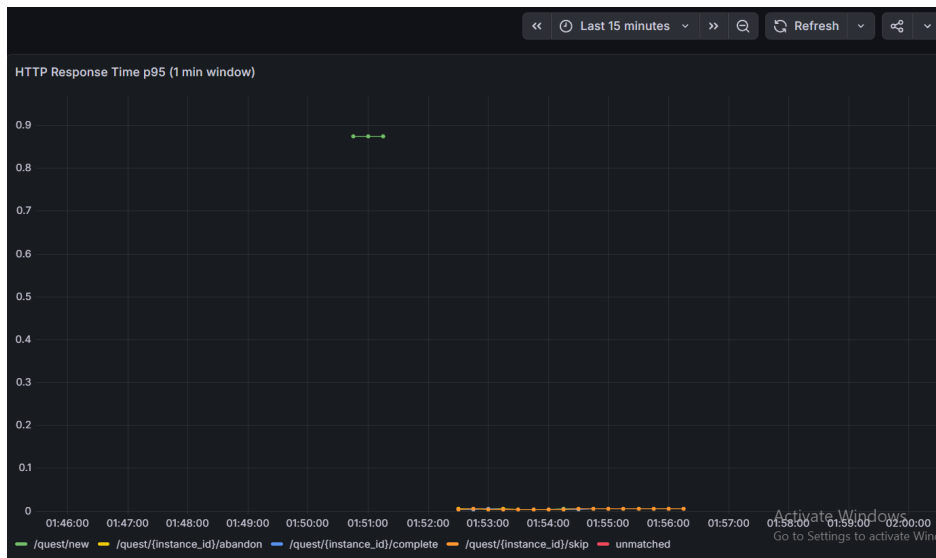
```
0.007577 0.026946 0.011364 0.004527 0.506021 (seconds)
```

The fourth request (the fifth cumulative call to the endpoint) shows almost exactly the 500 ms delay, confirming the fault fires as designed.

```
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> set FAULT_DELAY_MS=500
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> set FAULT_EVERY_N=5
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> docker compose up -d quest-app
[+] up 1/1
✓ Container quest-app Started 1.6s
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> curl -s -o nul -w "%{time_total}\n" http://127.0.0.1:8000/quest/new
0.007577
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> curl -s -o nul -w "%{time_total}\n" http://127.0.0.1:8000/quest/new
0.026946
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> curl -s -o nul -w "%{time_total}\n" http://127.0.0.1:8000/quest/new
0.011364
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> curl -s -o nul -w "%{time_total}\n" http://127.0.0.1:8000/quest/new
0.004527
(venv) C:\Users\DELL\Desktop\sem 5\ESD\assignment1\quest-app> curl -s -o nul -w "%{time_total}\n" http://127.0.0.1:8000/quest/new
0.506021
```

Terminal output of the five sequential curl timing checks showing the delayed request.

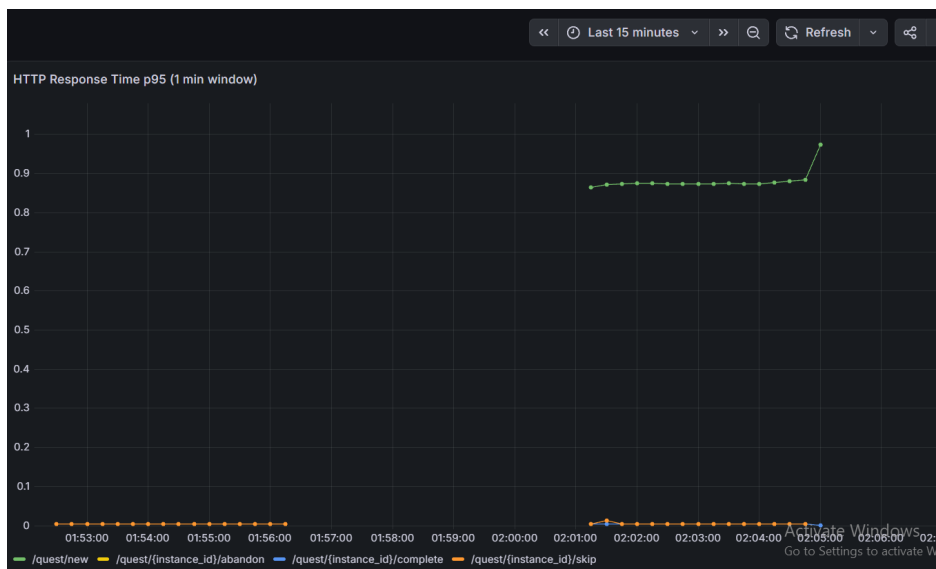
The same single delayed request is visible in Grafana moments later, as a brief isolated blip against an otherwise flat p95 line — confirming the manual test registered correctly before the full load test was run.



Grafana's HTTP p95 panel showing the single delayed request from the manual curl verification as a brief isolated spike.

Running the identical seeded load test under the fault (`--seed 100`, same duration/rate) produced the *same* 540 quests and outcomes (the seed controls the random decisions), but:

- Client-side average jumped from 16.8 ms to **111.6 ms**; p95 jumped from 30.0 ms to **521.6 ms**.
- Grafana's server-side p95 jumped sharply to a steady ≈ 0.87 – 0.88 s — matching the predicted ≈ 0.875 s almost exactly.



Grafana's HTTP p95 panel showing the sustained plateau at ≈ 0.85 – 0.9 s across the full seeded load test, once the fault was fully active.

Log evidence: the Kibana query `app.duration_ms >= 400` found **109 documents**, all concen-

trated exactly inside the fault-stage time window (a clean, isolated spike in Discover’s histogram, zero documents before or after). Two example documents showed `app.duration_ms`: 500.9 and 501.17, both on `endpoint`: `/quest/new`.



Explainable difference worth noting: the client-side p95 (521.6 ms) and Grafana’s server-side p95 (≈ 0.875 s) differ because they are computed differently — the load script only timed its own 540 end-to-end calls, while Grafana’s `histogram_quantile` interpolates across bucket boundaries over the full 1-minute rolling window of *all* traffic in that period. This is a real, explainable difference, not an error.

5.1.5 Stage 3: Recovery

Fault switched off (`FAULT_DELAY_MS=0`, `FAULT_EVERY_N=0`), verified with direct curl checks (six fast responses, 3–30 ms, no spikes). Running the identical seeded load test again gave the same 540 quests, with the average back down to **19.0 ms** and p95 **31.2 ms** — essentially matching the original baseline.

Confirming recovery: rather than chasing a single perfectly-timed screenshot of the p95 value decaying live in Grafana, recovery here is confirmed by two independent, solid measurements: the direct curl checks above (fast, no spikes) and the load-test script’s own client-side average dropping back to near-baseline. Together these give confidence that the fault’s effect had fully cleared, even without an additional screenshot of the transient decay itself.

5.1.6 E1 conclusion

A repeatable test (the same seeded load script) across three stages produced: baseline (≈ 30 ms client-side / ≈ 4 – 5 ms Grafana p95), fault (≈ 522 ms client-side / ≈ 875 ms Grafana p95, matching the bucket-interpolation prediction almost exactly), and recovery (back to ≈ 31 ms, matching baseline). Evidence spans metrics (Grafana panel screenshots, a direct Prometheus query), logs (a Kibana query with 109 matching documents concentrated exactly in the fault window, with two example values), and direct terminal proof (curl timings).

User impact: roughly 1 in 5 `/quest/new` calls took about half a second longer during the fault — noticeable, but not severe for this application’s use case.

5.2 E2: Cardinality Explosion (5 marks)

5.2.1 Setup

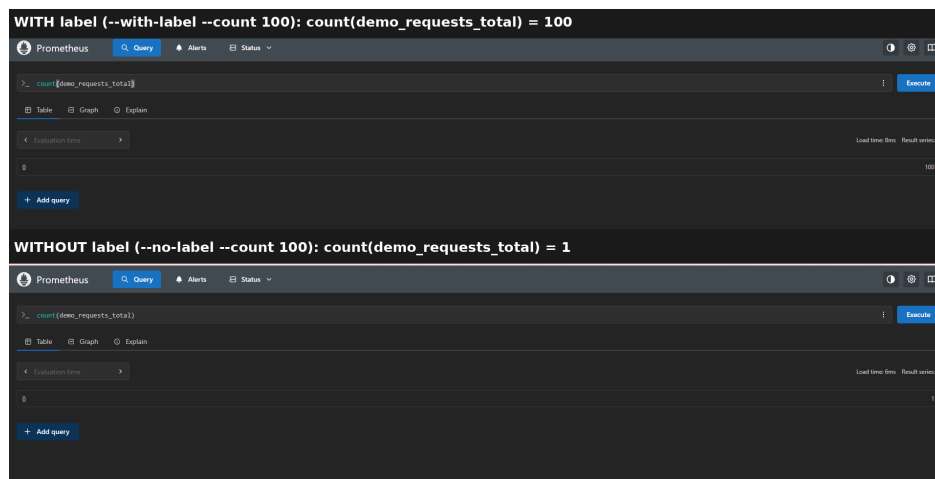
A standalone script, `scripts/cardinality_demo.py`, deliberately kept separate from the real quest-app metrics, runs its own tiny Prometheus client HTTP server on port 9091:

- `--with-label` mode: a Counter `demo_requests_total` with a `request_id` label, unique per simulated request — **the bad practice being demonstrated**.
- `--no-label` mode: the same counter, with no label — **the good practice**.

A temporary fourth Prometheus job (`cardinality-demo`, targeting `host.docker.internal:9091`) was added to scrape it, verified UP, and later **removed** after the experiment was complete, leaving the final `monitoring/prometheus.yml` clean with just the `quest-app` and `node-exporter` jobs.

5.2.2 Results

- **With-label run:** `python scripts\cardinality_demo.py --with-label --count 100`. Query `count(demo_requests_total)` → **100** (one time series per unique `request_id`).
- **No-label run:** same 100 simulated requests, no label. Query `count(demo_requests_total)` → **1** (a single series `{}`).



Side-by-side comparison: 100 series with the label vs. 1 series without.

Core result: 100 requests with a per-request unique label created 100 separate time series. The exact same 100 requests with no unique label created just 1 time series. **Note on stored history:** the assignment’s own guidance points out that removing a label does not immediately delete stored history — the 100 old series from the `--with-label` run remain in Prometheus’s storage as stale time series. They stop appearing in *new* instant queries once Prometheus marks them stale (after roughly 5 minutes without a fresh sample), which is why `count(demo_requests_total)` correctly shows 1 after switching modes — but the disk space and WAL entries for those 100 old series are not reclaimed until Prometheus’s normal compaction process runs.

5.2.3 Cost-at-scale explanation

Every unique label *combination* creates its own time series in Prometheus, each with its own memory, disk, and index cost (a chunk of memory for the in-memory head block, plus entries in the on-disk TSDB and WAL). An unbounded-cardinality label — a request ID, session ID, or raw URL — means series count grows without limit as real traffic scales up, causing memory bloat, disk growth, and slower queries over time.

Tie-back point: this is exactly the principle the real app’s own **endpoint** label already follows correctly (route template, not raw URL, per Section 4.1) — E2 demonstrates in isolation the exact failure mode the app was already designed to avoid.

Why identifiers belong in logs instead: Elasticsearch indexes field *values* without needing to maintain a persistent time series per unique value the way Prometheus does — so a unique `request_id` or `instance_id` is entirely appropriate as a **log field** (which the app already does correctly), just not as a **Prometheus label**.

No attempt was made to actually crash Prometheus with this experiment — 100 series was judged a safe, sufficient, illustrative scale for the report.

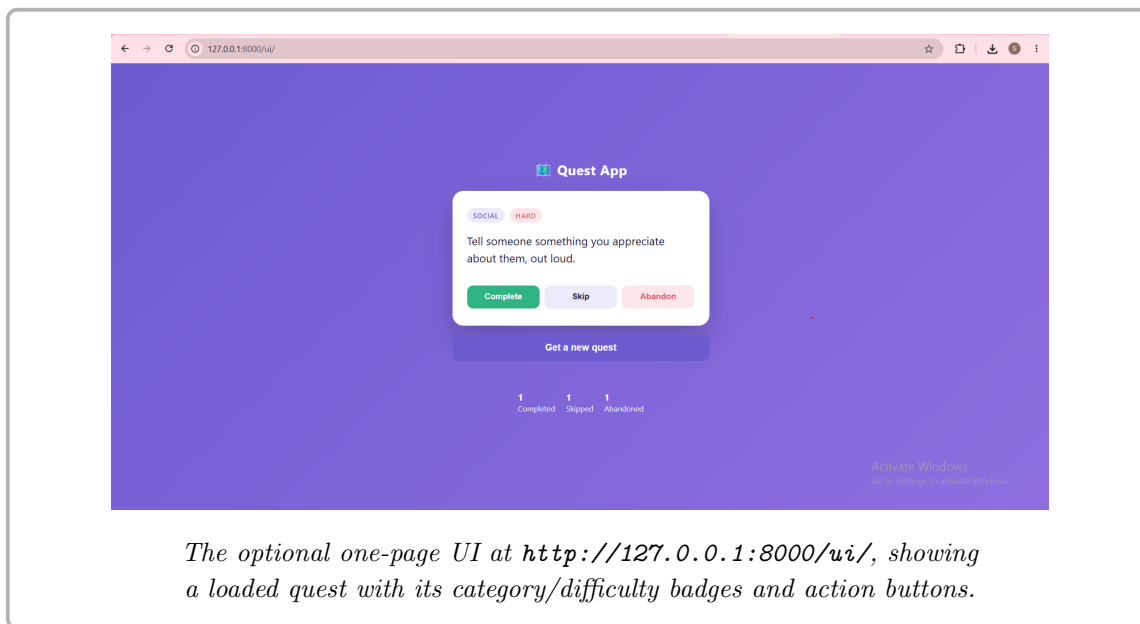
6 Optional Addition: One-Page Web UI

Not required by the assignment (Part A carries no UI requirement), but added as a small extra on top of the core API.

- **Files:** `app/static/index.html` — a self-contained single-page UI (inline CSS and JavaScript, no external dependencies): a quest card with category/difficulty badges, Complete/Skip/Abandon buttons, a “Get a new quest” button, and a client-side-only session tally.
- `app/main.py` **change** — only two lines added:

```
from pathlib import Path
from fastapi.staticfiles import StaticFiles
...
app.mount("/ui", StaticFiles(directory=Path(__file__).parent / "static",
                                     html=True), name="ui")
```

- **How it works:** the page calls the existing API endpoints directly via `fetch()` — no new backend endpoints, no new dependencies (FastAPI’s `StaticFiles` is built in).



The optional one-page UI at `http://127.0.0.1:8000/ui/`, showing a loaded quest with its category/difficulty badges and action buttons.

Report note: the `/ui/` page’s own HTTP traffic (loading the HTML, its JavaScript calling `/quest/new`, etc.) flows through exactly the same middleware, metrics, and logging as any other client — no special-casing was added for it.

7 README, Cleanup, and Submission

7.1 Starting and using the project

```
docker compose up -d
```

Elasticsearch and Kibana take a minute or two to start; other services are fast. See the project's README.md for the full list of URLs, curl examples, and how to run the load generator, the fault switch, and the cardinality demo.

7.2 Safely cleaning up

- **Pause everything** (keeps all data): `docker compose stop`
- **Do NOT run `docker compose down` casually** — Elasticsearch's logs survive this (named volume `esdata`), but **Grafana and Prometheus have no volumes, so their dashboards and metric history are lost**. The Grafana dashboard is exported to `monitoring/dashboards/*.json` specifically as a backup for this reason.
- `docker compose down -v` additionally deletes the `esdata` volume, permanently removing all stored logs. The 7-day retention setting must be re-applied afterwards using the command in Part C, Section 3, since the setting itself does not survive the volume's deletion.

7.3 Submission checklist

- Code and config: `app/` (including `app/static/`), `Dockerfile`, `docker-compose.yml`, `requirements.txt`, `monitoring/` (including the dashboard JSON), `logging/` (including the Kibana queries file).
- README.md covering start, use, test, and safe cleanup.
- This report, covering Parts A through E.
- **Excluded:** `venv/`, `__pycache__/`, `.git/`, data volumes.

Creating the final zip (run inside the `quest-app` folder):

```
git archive -o ../quest-app-submission.zip HEAD
```

This includes only git-tracked files from the latest commit, so `venv/`, `.git/`, and `__pycache__/` are excluded automatically. Verify with:

```
cd ..  
tar -tf quest-app-submission.zip
```

7.4 AI assistance

[**TODO — fill in per the exact course policy.**] AI assistance (Claude) was used throughout this project for: explaining Prometheus/Grafana/Elasticsearch/Kibana/Filebeat concepts; suggesting PromQL and KQL queries; drafting boilerplate code (the JSON log formatter, the HTTP middleware, the load-testing and cardinality-demo scripts, the optional UI); and helping structure this report from working notes kept throughout the project. All commands were run and verified by the student personally; all reported results (counts, timings, screenshots) reflect the student's own running system.